v1.0.1
20260226

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[module 1.2]

MODELLING CONCURRENT PROGRAM EXECUTION

SUMMARY

- Making models of concurrent programs
  - State diagrams & execution scenarios
- Correctness
  - safety and liveness properties
  - fairness
- Critical Section Problem
  - path towards Dekker's algorithm
- Verification of Concurrent Programs
  - linear temporal logic - LTL

FROM PROGRAMS TO MODELS (AND BACK)

- Importance of models and abstraction for computer science and engineering in particular
  - model: rigorous description / representation of program (system) structure and behaviour at a proper level of abstraction
    - including relevant information, abstracting from non-relevant aspects
  - diagrammatical representations for program design
  - formal models for program analysis and verification
- Defining proper models for concurrent programs
  - defining models for the structure and behaviour of concurrent programs abstracting from the low-level details of their actual implementation and realisation
    - design progettazione
  - enabling the possibility to reason about their dynamic behaviour of concurrent programs
    - verification (Verificare che il programma sia corretto o meno)

A MODEL FOR CONCURRENT PROGRAM EXECUTION

- Modelling each process as a sequence of **atomic actions**, each action corresponding to the atomic execution of a statement
- ***Speed independence assumption*** **=>** L'ipotesi di indipendenza dalla velocità presuppone che le velocità relative dei processi siano sconosciute e potenzialmente variabili, purché siano positive (ovvero, i processi procedano).

  modelling the execution of a concurrent program as a sequence of actions obtained by alternando arbitrariamente ~~**arbitrarily interleaving**~~ the actions (atomic statements) from the processes Poiché non vengono formulate ipotesi sulla velocità di esecuzione dei processi, è sicuro modellare i programmi concorrenti alternando arbitrariamente le azioni (istruzioni atomiche) di ciascun processo.
  - a single abstract *global processor* executing all the actions
  - **atomic** statements => executed fino al ~~to~~ completion without the possibility of interleaving (devo eseguire fino alla fine, quando inizio, tutto lo statement atomico, senza interleaving di altri processi: Un'azione è atomica se, una volta iniziata, non può essere interrotta.)
  - during the computation the *control pointer* or instruction of a process indicates the next statement that can be executed by that process
- a **computation** or **scenario** is an execution sequence that can occur as a result of the interleaving Siccome non sappiamo quale processo sarà più veloce (per l'assunzione di speed independence), la "storia" di ciò che accade nel computer è solo una delle tante combinazioni possibili. Dato che il processore può scegliere arbitrariamente quale azione eseguire tra quelle indicate dai control pointer dei processi, l'esecuzione di un programma concorrente non è una linea retta, ma un albero di possibilità: Ogni ramo dell'albero è un'esecuzione sequenziale (uno scenario). Ogni nodo rappresenta lo stato della memoria e la posizione dei puntatori d'istruzione in quel momento. Se stai progettando un sistema concorrente, devi assicurarti che tutti i possibili scenari (tutte le sequenze di interleaving) portino a un risultato corretto. Se esiste anche un solo scenario su un milione in cui l'interleaving causa un errore (ad esempio, entrambi i processi scrivono sulla stessa variabile contemporaneamente), allora il tuo programma ha un bug di concorrenza.

FIRST TRIVIAL EXAMPLE

Due threads/processi che hanno memoria condivisa e locale

integer n := 0	
p	**q**
integer k1 := 1 **p1:** n := k1	integer k2 := 2 **q1:** n := k2

- Each labeled line represents an atomic statement
- Each process has private memory
  - local variables, such as k1 and k2
- Processes shares some memory
  - global variables, such as n
- Program execution: 2 scenarios
  - p1, q1 (=> n finally is equal to 2) prima p1, poi q1
  - q1, p1 (=> n finally is equal to 1) prima q1, poi p1

Uno scenario è una particolare "linea temporale" (sequenza di esecuzione di istruzioni interleaved di ogni istruzione di ogni processo che segue l'odine implicito di un processo) generata mescolando le istruzioni dei vari processi. Studiare la concorrenza significa analizzare tutti questi possibili scenari per assicurarsi che nessuno di essi rompa il sistema.

STATE DIAGRAMS

- Given the model, the execution of a concurrent program can be formally represented by **states** and **transitions** between states
  - the state is defined by a tuple consisting of
    - one element of each process that is a label (statement) from that process
    - one element for each global or local variable that is a value whose type is the same as the type of a variable
  - there is a transition between two states s1 and s2 if executing a statement in state s1 changes the state to s2
    - the statement executed must be one of those pointed to by a control pointer in s1
- The **state diagram** is a *graph* containing all the *reachable states* of the programs
  - scenarios are represented by directed paths through the state diagram from the initial state
  - cycles represent the possibility of infinite computation in a finite graph

FIRST EXAMPLE: STATE DIAGRAM

a che istruzione sono nel processo p?

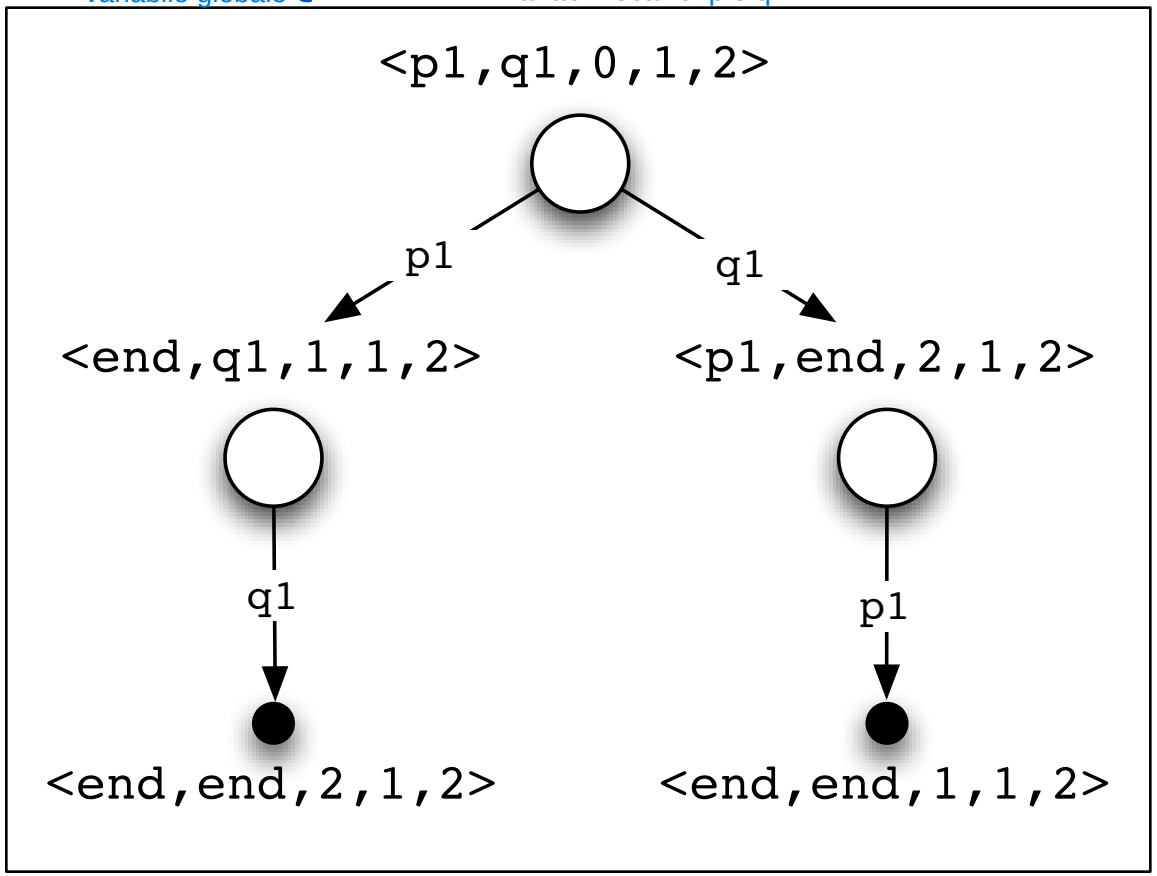
a che istruzione sono nel processo q?

- State tuple: <p,q,n,k1,k2>

variabile globale

variabili locali di p e q

Un Labeled Transition System (LTS) è proprio il formalismo matematico usato per descrivere quello che abbiamo chiamato "diagramma di stato" di un programma concorrente.



Transizioni sono etichettate dalle istruzioni che scelgo

- 5 states, 2 scenarios

Nella pratica, non tutti gli scenari sono equiprobabili

EXAMPLE #2

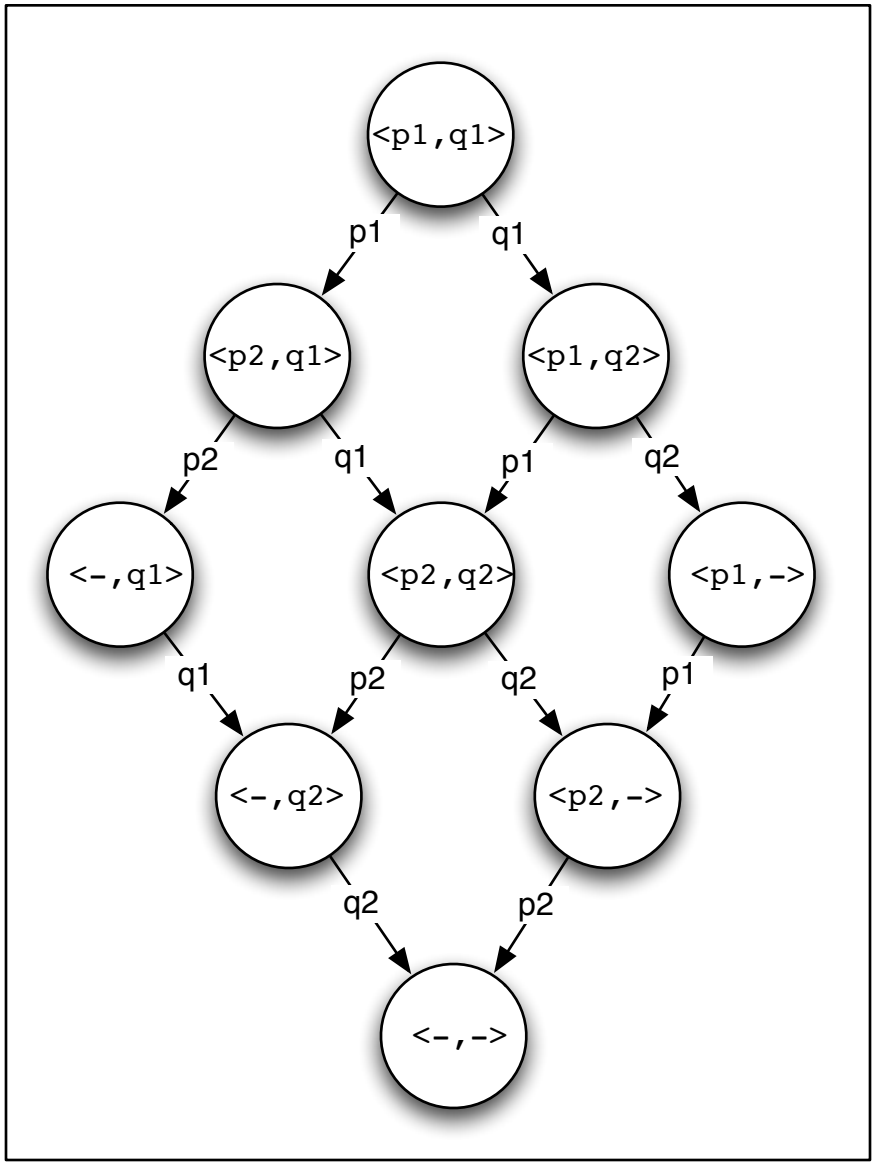
p	q
p1: `print("p1")` **p2:** `print("p2")`	**q1:** `print("q1")` **q2:** `print("q2")`

- State diagram?
- How many scenarios?

EXAMPLE #2

- 6 scenarios:

```
p1 p2 q1 q2
p1 q1 p2 q2
p1 q1 q2 p2
q1 q2 p1 p2
q1 p1 q2 p2
q1 p1 p2 q2
```



EXAMPLE #3

p	q
p1: `print(“p1”)` **p2:** `print(“p2”)` **p3:** `print(“p3”)`	**q1:** `print(“q1”)` **q2:** `print(“q2”)` **q3:** `print(“q3”)`

- Scenarios?

EXAMPLE #3

- 20 scenarios:

```
p1 p2 p3 q1 q2 q3
p1 p2 q1 p3 q2 q3
p1 q1 p2 p3 q2 q3
q1 p1 p1 p3 q2 q3
p1 p2 q1 q2 p3 q3
p1 q1 p2 q2 p3 q3
q1 p1 p2 q2 p3 q3
p1 q1 q2 p2 p3 q3
q1 p1 q2 p2 p3 q3
q1 q2 p1 p2 p3 q3
p1 p2 q1 q2 q3 p3
p1 q1 p2 q2 q3 p3
q1 p1 p2 q2 q3 p3
p1 q1 q2 p2 q3 p3
q1 p1 q2 p2 q3 p3
q1 q2 p1 p2 q3 p3
p1 q1 q2 q3 p2 p3
q1 p1 q2 q3 p2 p3
q1 q2 p1 q3 p2 p3
q1 q2 q3 p1 p2 p3
```

NUM. SCENARIOS WITH 2 PROCESSES

num. actions per proc.	num. scenarios
1	2
2	6
3	20
4	70
5	252
6	924
7	3432
8	12820
...	....

- Note: in these cases process actions have no dependencies...

EXAMPLE #4

- Changing the number of processes: 3 processes..

p	**q**	**r**
p1: print(“p1”) **p2:** print(“p2”)	**q1:** print(“q1”) **q2:** print(“q2”)	**r1:** print(“r1”) **r2:** print(“r2”)

- Scenarios?

EXAMPLE #4

- 90 scenarios

```
p1 p2 q1 q2 r1 r2
p1 p2 q1 r1 q2 r2
p1 p2 q1 r1 r2 q2
p1 p2 r1 q1 q2 r2
p1 p2 r1 q1 r2 q2
p1 p2 r1 r2 q1 q2
p1 q1 p2 q2 r1 r2
...
q1 p1 p2 q2 r1 r2
q1 p1 p2 r1 q2 r2
q1 p1 p2 r1 r2 q2
q1 p1 q2 p2 r1 r2
q1 p1 q2 r1 p2 r2
...
r1 p1 p2 q1 q2 r2
r1 p1 p2 q1 r2 q2
r1 p1 p2 r2 q1 q2
r1 p1 q1 p2 q2 r2
r1 p1 q1 p2 r2 q2
...
r1 r2 q1 p1 q2 p2
r1 r2 q1 q2 p1 p2
```

NUM. SCENARIOS WITH 3 PROCESSES

num. actions per proc.	num. scenarios
1	6
2	90
3	1680
4	34650
...	...

GENERALIZING...

La formula serve a calcolare esattamente quanti sono i possibili scenari (le diverse sequenze di esecuzione) che possono generarsi mescolando le istruzioni di più processi.

- Number of scenarios produced by n processes, each having m_i actions:

numero di processi totali

numero di azioni atomiche nel processo i-esimo.

$$ns = \frac{(\sum_{i=1}^{n} m_i)!}{\prod_{i=1}^{n} (m_i!)}$$

rappresenta le permutazioni totali di tutte le istruzioni come se fossero tutte distinte.

questa è la formula delle permutazioni con ripetizione.

serve a eliminare le combinazioni impossibili, perché all'interno di ogni singolo processo l'ordine delle istruzioni è fisso (l'azione 1 deve sempre avvenire prima dell'azione 2 dello stesso processo).

Esplosione degli Stati

	$n = 2$	3	4	5	6
$m_i = 2$	6	90	2520	113400	$2^{22.8}$
3	20	1680	$2^{18.4}$	$2^{27.3}$	$2^{36.9}$
4	70	34650	$2^{25.9}$	$2^{38.1}$	$2^{51.5}$
5	252	$2^{19.5}$	$2^{33.4}$	$2^{49.1}$	$2^{66.2}$
6	924	$2^{24.0}$	$2^{41.0}$	$2^{60.2}$	$2^{81.1}$

"THE IMPORTANCE OF BEING ATOMIC"

Stesso esempio, ma due tipi di atomicità a livello di istruzione

- Atomic increment (1)

integer n := 0	
p	**q**
p1: n := n + 1	**q1:** n := n + 1

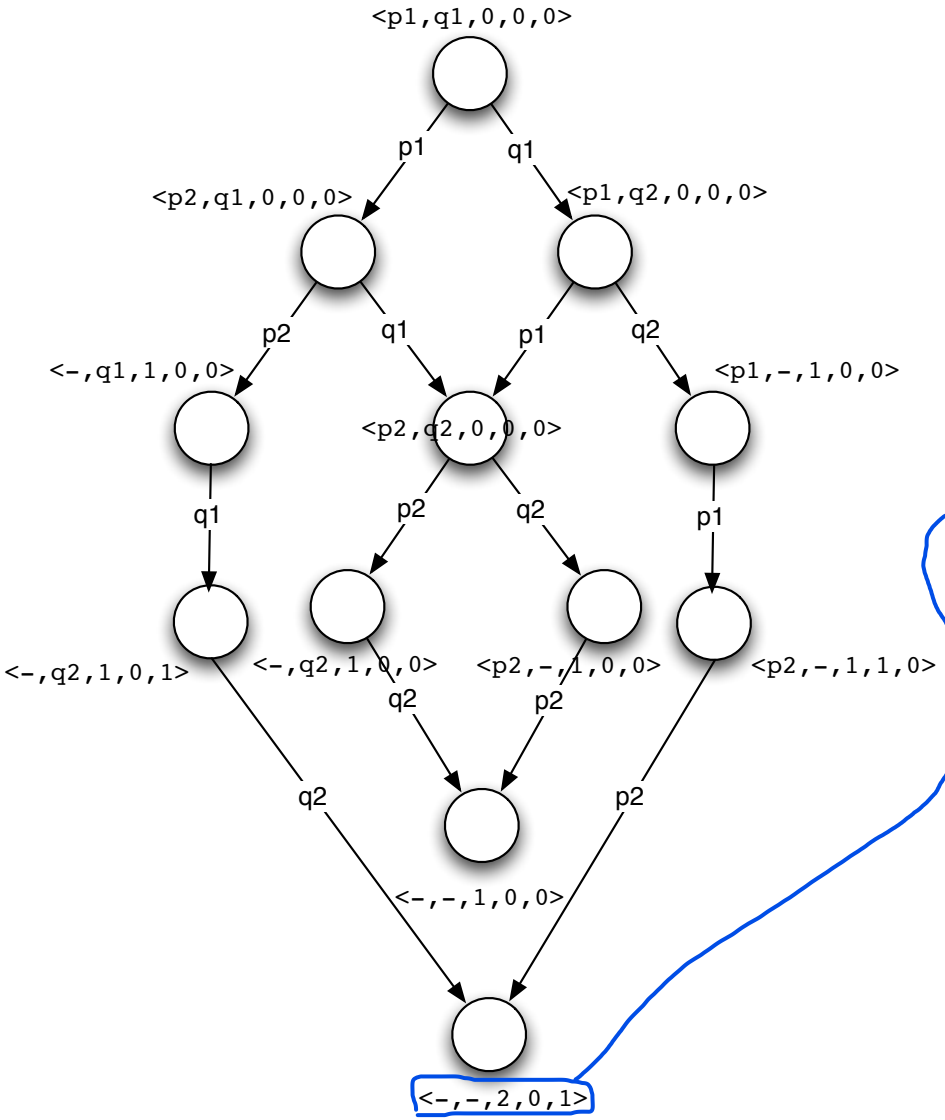
- Non-atomic increment (2)

integer n := 0	
p	**q**
integer tmp; **p1:** tmp := n **p2:** n := tmp + 1	integer tmp; **q1:** tmp := n **q2:** n := tmp + 1

sono due var. tmp diversi

- In the second case, scenarios exist in which the final value of n is 1 ad esempio, p1 q1 p2 q2 (perchè in tmp ho 0, entrambi metto n=1 ed il risultato è 1: in realtà, dovrebbe essere = 2 come sopra)

NON ATOMIC CASE: STATE DIAGRAM



12 states

Only 2 scenarios over 6 have final n value = 2

Un unico stato dove due scenari arrivano a quello stato

Quindi, si costruisce il grafo e poi capire se valgono le proprietà di correttezza

ASSIGNMENTS & INCREMENTS AT THE MACHINE-CODE LEVEL

- Stack machines

integer n := 0	
p	**q**
p1: push n **p2:** push #1 **p3:** add **p4:** pop n	**q1:** push n **q2:** push #1 **q3:** add **q4:** pop n

- Register machines

integer n := 0	
p	**q**
p1: load R1, n **p2:** add R1,#n **p3:** store n, R1	**q1:** load R1, n **q2:** add R1,#n **q3:** store n, R1

non tutti i dati vengono letti/modificati allo stesso modo per il processore: alcuni vengono gestiti come un blocco unico, altri come un puzzle di pezzi separati.

NON-ATOMIC STRUCTURES (1/3)

- The notion of “atomic” can be referred not only to actions, but also to data structures:
  - a data object is defined *atomic* if it can be in a finite number of states equals to the number of values that it can assume
    - operations change (atomically) that state

Un oggetto dati è atomico se il passaggio da un valore all'altro avviene istantaneamente, senza che nessuno possa vedere un valore "a metà".

Se una variabile può valere 0 o 1, e io la cambio, essa passa da 0 a 1 senza mai passare per stati intermedi strani.

  - typically primitive data type in concurrent languages are atomic
    - not always: e.g. `double` in Java
- Abstract data types composed by multiple simpler data objects are typically non atomic
  - es: class in OO languages, structs in C

Una sequenza di istruzioni può essere considerata atomica se, anche se ci può essere interleaving, osservandola/analizzandola, essa non porta a nessun tipo di interferenza o problema ad equipararlo ad un'istruzione atomica

NON-ATOMIC STRUCTURES (2/3)

- In that case for the ADT (or more generally data object) it is possible to identify two basic types of states: *internal* and *external*
  - Lo Stato Interno è la configurazione reale di tutti i pezzetti (variabili, bit, puntatori) che compongono l'oggetto in memoria.
  - the internal state is meaningful for who defines the data object (class)

    Durante un'operazione (es. un bonifico), l'oggetto deve modificare diverse variabili una alla volta. In quei micro-istanti, lo stato interno dell'oggetto cambia continuamente.
  - Lo Stato Esterno è l'insieme dei valori che hanno un senso logico per chi usa l'oggetto.
  - the external state is meaningful for who uses the data object

    Immagina un oggetto ContoBancario. Gli stati esterni validi sono quelli in cui il saldo riflette operazioni concluse (es. " Ho 100€" oppure "Ho 50€"). Chi usa la classe si aspetta che l'oggetto sia sempre in uno di questi stati coerenti.
- The correspondence among internal and external states is *partial*

  Significa che esistono molti stati interni che non corrispondono a nessuno stato esterno valido. Questi stati sono definiti incoerenti (non consistent).
  - there exist internal states which have no a correspondent external state
  - internal states which have a correspondent external state are defined **consistent**

    Uno stato interno è consistente solo se "ha un corrispondente stato esterno". Ovvero: se un osservatore esterno guarda l'oggetto in quel momento, vede qualcosa che ha senso e che rispetta le regole del programma.

Il problema delle strutture non atomiche è che mettono a nudo i loro stati interni incoerenti.
Se l'oggetto fosse atomico, passerebbe da "Stato Esterno 1" a "Stato Esterno 2" in un colpo solo.
Essendo non atomico, l'oggetto "si apre", mostra le sue viscere (stati interni) e, se sfortunatamente un altro processo guarda proprio in quel momento, il sistema fallisce perché legge dati che non dovrebbero esistere.

NON-ATOMIC STRUCTURES (3/3)

- Then, the execution of an operation on a (not-atomic) ADT can go through states that are *not consistent* (es: oggetti standard non sono Thread-safe)
  - e.g. a simple list
- This is not a problem in the case of sequential programming
  - thanks to information hiding
- Conversely, it is a problem in the case of concurrent programming
  - it can happen that a process would work on an object while the object is in an inconsistent state, since an process is concurrently operating on it

> it is necessary to introduce proper mechanisms that would guarantee that processes work on data objects that are always in states that are consistent (ADT Thread-safe: permette di eseguire su di esse delle istruzioni in modo atomico)

CYCLIC PROCESSES

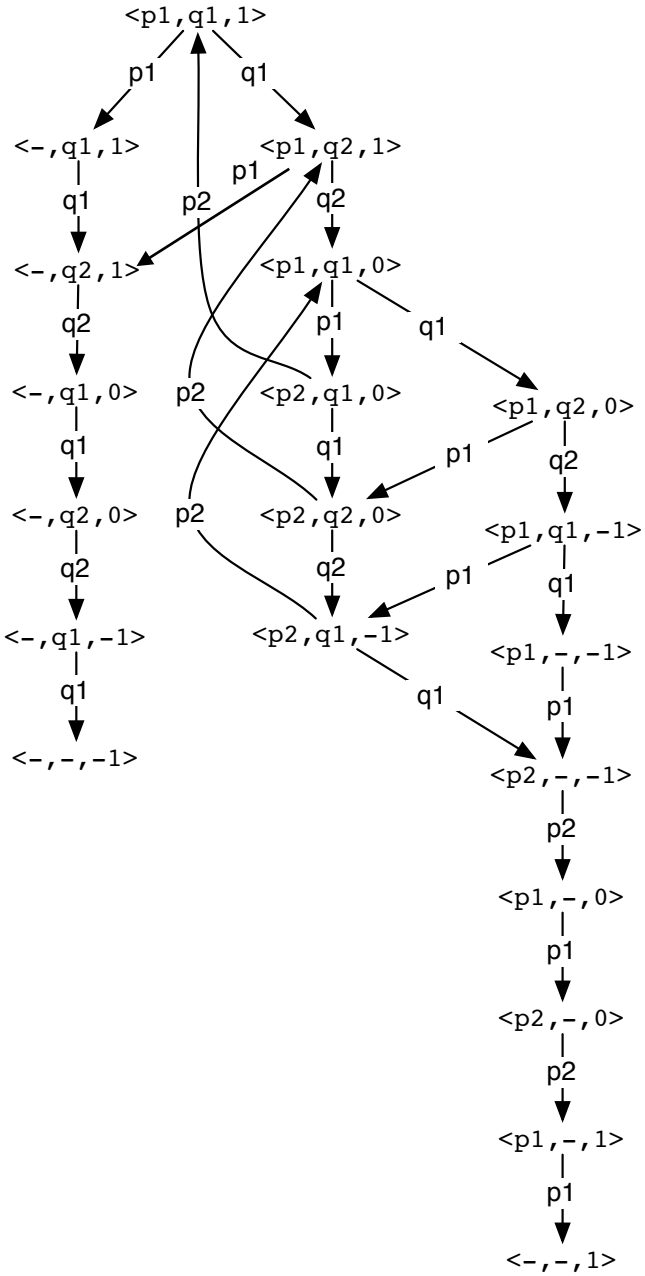
- **p** and **q** processes cycling on a condition

integer n := 1	
p	**q**
p1: while (n < 1) **p2:** n := n + 1	**q1:** while (n >= 0) **q2:** n := n - 1

Numero di stati finito
Numero di transizioni/istruzioni finito

- Analysis
  - state diagram ?
  - construct a scenario in which the loop in p executes exactly one
  - construct a scenario in which the loop in p executes exactly three times

STATE DIAGRAM



IS THIS MODEL A *GOOD* MODEL ?

OR RATHER: IS THE CONCURRENT PROGRAMMING ABSTRACTION JUSTIFIABLE ?

- Actually in the reality computer system **has not a global state**
  - matter of physics
- That's the the role of abstraction: *we create a model of the system in which a kind of global entity executes the concurrent program by arbitrarily interleaving statements*
  - to ease analysis
- But.... is it a valid model for real concurrent computing systems? => *reality check* Questo modello interleaved è ancora valido in tutti e 4 i tipi di sistemi
  - multitasking systems
  - multicore systems
  - multiprocessor computers
  - distributed systems

ARBITRARILY INTERLEAVING: ABSTRACTING FROM TIME

- **Arbitrary interleaving** means that we ignore time in our analysis of concurrent programs
  - focussing only to
    - partial orders related to action sequences $a_1, a_2, \ldots$
    - atomicity of the individual action a_j => choosing what is atomic is fundamental
  - robustness with respect to both hardware (processor) and software changes
    - independent from changes in timings / performance
- This makes concurrent programs amenable to *formal analysis*, which is necessary to ensure **correctness** of concurrent programs.
  - proving correctness besides the actual execution time, which is typically strictly dependent on processors speed and system's environment timings

CORRECTNESS OF PROGRAMS

- Checking correctness for sequential programs
  - unit testing based on specified **input** and expecting some specified **output**
    - diagnose, fix, rerun cycle
  - re-running a program with the same input will always give the same result
- Concurrent programming new (challenging) perspective
  - **the same input can give different outputs**, depending on the scenario...
    - some scenarios may give correct output while others do not
  > we can’t debug a concurrent program in the normal way
    - each time you run the program we will likely get a different scenario
- Needs of different kind of approaches
  - formal analysis, *model checking*
  - based on abstract models

I Test, in un programma concorrente, non bastano: perchè ad ogni esecuzione di un test viene scelto uno dei scenari possibili di esecuzione che dipendono dall'architettura del calcolatore, situazione in cui si trova il sistema operativo etc...

CORRECTNESS OF CONCURRENT PROGRAMS

- The correctness of (possibly non-terminating) concurrent programs is defined in terms of **properties** of computations
  - conditions that must be verified in every possible scenarios
- Two type of correctness properties
  - **safety** property
  - **liveness** property

SAFETY PROPERTIES

- The property must be **always** true
  - i.e. for a safety property P to hold, it must be true in *every state* of *every computation*
  - expressed as *invariants* of a computations
- Typically used to specify that "bad things" should never happen
  - mutual exclusion
    - no more than one process is ever present in a critical region (vado a controllare in ogni stato se in ogni sezione critica ci sono più di uno stato)
  - no deadlock
    - no process is ever delayed awaiting an event that cannot occur (non ci devono essere stati di deadlock: stato da cui non posso più eseguire transizioni perché non ce ne sono)
  - ...

Una proprietà P è di Safety se P è vera per ogni possibile stato

LIVENESS (OR *PROGRESS*) PROPERTIES

Prima o poi, in ogni stato quella condizione/proprietà si verifica (es: JUnit Testing). Non basta vedere solo gli stati, ma bisogna verificare tutti i possibili scenari che prima o poi quella proprietà diventi vera.

- The property must **eventually** *become true*
  - i.e. for a liveness property P to hold, it must be true that *in every* computation there is some state in which P is true
- Typically used to specify that "good things" eventually happen
  - no starvation
    - a process finally gets the resource it needs (CPU time, lock)
  - no dormancy
    - a waiting process is finally awakened
  - reliable communication
    - a message sent by one process to another will be received
  - ...

FAIRNESS

Riguarda come nel nostro sistema vengono schedulate le azioni

- A liveness property which holds that something good happens infinitely often
- Main example
  - a process activated infinitely often during an application execution, each process getting a fair turn
  - i.e. an action that can be executed, eventually will be executed
    - requirement on the scheduling
- So programs can have different liveness behavior depending on precisely how their instructions are interleaved
  - how instructions are interleaved is a result of a **scheduling policy.**

FAIRNESS & SCHEDULING POLICIES

- **Unconditional Fairness**
  - a scheduling policy is unconditionally fair if every unconditional atomic action that is eligible is executed eventually
- **Weak Fairness**
  - a scheduling policy is weakly fair if it is unconditionally fair and every eligible *conditional* atomic action whose condition becomes and remains true is executed eventually
- **Strong Fairness**
  - a scheduling policy is strongly fair if it is unconditionally fair and every eligible conditional atomic action whose condition becomes true *infinitely often* (infinitely many times) is executed eventually

UNCONDITIONALLY FAIR SCENARIO

- def. **unconditionally fair scenario**
  - a *scenario* is *(unconditionally) fair* if at any state in the scenario a statement that is *continually enabled* eventually appears in the scenario

```
integer n := 0
boolean flag := false
```

p	q
p1: while flag = false **p2:** n := 1 - n	**q1:** flag := true

- Does this algorithm necessarily halt?
  - yes if we assume only fair scenarios
    - if we allow only fair scenario, then eventually an execution of q1 must be included in every scenario
  - **the non-terminating scenario is *not* fair**

Esempio: in un sistema distribuito non viene mai eseguito q perché è down

CRITICAL SECTION PROBLEM

- Introduced by Dijkstra in 1965, the critical section problem can be considered one of the most important and well-studied problem in concurrent programming, in the context of process competition
- Problem definition
  - N processes, each executing in an infinite loop a sequence of statements that can be divided in 2 subsequences: the *critical section* (CS) and the *non-critical section* (NCS)

istruzioni

```
P[1..n]:
loop {
   <non-critical section>
   entry (or pre-) protocol
   <critical section>
   exit (or post-) protocol
   <non-critical section>
}
```

Insieme di istruzioni da fare prima di entrare in sezione critica (affinché sia una sezione critica)

Insieme di istruzioni da fare dopo essere usciti dalla sezione critica (affinché sia una sezione critica)

Quando un processo è in sezione critica, gli altri processi posso fare qualsiasi cosa tranne essere nella propria sezione critica (solo un processo alla volta in sezione critica)

istruzioni

  - each critical section is typically a sequence of statements that access some shared object.

CRITICAL SECTION PROPERTIES

Proprietà della Sezione Critica per essere tale

- Our task is to design *entry and exit protocols* that satisfy the following properties:
  - **mutual exclusion** (un processo alla volta nella propria sezione critica)
    - statements from the critical sections of two or more processes must not be interleaved
  - **freedom from deadlock**
    - if some processes are trying to enter their critical sections, then one of them must eventually succeed
  - **freedom from individual starvation**
    - if any process tries to enter its critical section, then that process must eventually succeed (prima o poi quel processo entrerà nella propria sezione critica)
    - *bounded waiting* property
- Any proposed solution must satisfy also the *progress property* for the ~~CS~~ Critical Section

  prima o poi, si esce dalla sezione critica
  - once a process starts to execute the statements of its critical section, it must eventually finish.
- The ~~NCS~~ Non Critical Section need not to progress (posso rimanere all'infinito in NCS: quindi non vale la progress property per NCS)
  - we can have infinite loops or termination outside the ~~CS~~ Critical Section

CONCRETE PROBLEMS BASED ON CRITICAL SECTIONS

- The critical section problem is intended to model a system that performs complex computation, but occasionally need to access data or resources (e.g. hardware) that are shared by several process.
- Example
  - a check-in-kiosk at an airport, accessing to a central database of passengers

CRITICAL SECTION SOLUTION: ALGORITHMS VS. MECHANISMS

- The CS problem can be easily solved by using basic mechanisms such as *semaphores* or *locks*, that must be provided by the concurrent machine

  Meccanismi di alto livello che rendono, insieme di istruzioni non-atomiche, atomiche (che semplificano la soluzione rispetto a quella puramente algoritmica)

  – next modules
- In this module we focus instead on a purely algorithmic solution (Soluzione articolata)
  – without using high-level mechanism
  – using solely basic atomic statements (e.g. load and store)

  Costrutti sequenziali di base (while, if, ..) ed Assegnamento Atomico di variabili

CS FOR TWO PROCESSES

Ricorda assunzione: istruzioni etichettate con etichetta vengono eseguite atomicamente

- First we will analyse the problem considering 2 processes P and Q sharing some global variables

Critical section problem	
Global variables	
p	q
Local variables loop forever non-critical section pre-protocol critical section post-protocol	Local variables loop forever non-critical section pre-protocol critical section post-protocol

- The protocols may require local or global variables
  - we assume that no variables used in the CS and NCS are used in the protocols

FIRST ATTEMPT

- Let's consider a first possible solution

First attempt	
Integer turn ← 1	
p	q
loop forever p1: *non-critical section* p2: await turn = 1 Pre-Protocol di p p3: *critical section* p4: turn ← 2 Post-Protocol di p	loop forever q1: *non-critical section* q2: await turn = 2 Pre-Protocol di q q3: *critical section* q4: turn ← 1 Post-Protocol di q

await sarebbe: while (turn != 1)

- The statement **await** turn = 1 is an implementation independent notation for a statements that waits until the condition turn = 1 becomes true.
  - this can be implemented (albeit inefficiently) by a busy-wait loop that does nothing until the condition is true.

1 CHECKING CORRECTNESS

- In order to prove the correctness of the solution we construct the state diagram of the concurrent program
  - the algorithm must satisfy the three properties
- To check correctness properties is only necessary to examine the set of reachable states and the transitions among them
  - states are triple $<p_i, q_j, turn>$ where:
    - p_i = current control pointer for process P
    - q_j = current control pointer for process Q
    - `turn` = content of turn variable
  - the diagram is constructed incrementally starting from the initial state and considering what the potential states are
    - 16 states
- Describing the mutual exclusion property (non deve esserci uno o più stato in cui ho sia p3 sia q3 contemporaneamente, a prescindere dal valore di turn)
  - the mutual exclusion correctness property holds if the set of all accessible states does not contain a state of the form $<p_3, q_3, _>$.

REDUCING THE NUMBER OF STATES

- For studying correctness, we can reduce the number of states without altering the semantics of the program
  - by removing unessential statements
- Whatever statements are executed by the CS and in the NCS are totally irrelevant to the correctness of the synchronisation algorithms, so they can be removed (vengono lasciati i Protocolli Pre e Post CS)

Semplifico il modello rimuovendo le istruzioni non essenziali

First attempt (abbreviated)	
Integer turn ← 1	
p	q
loop forever p1: await turn = 1 p2: turn ← 2	loop forever q1: await turn = 2 q2: turn ← 1

Anche qui, ho starvation (es: scelgo solo q1 e così p non entra mai in CS)

- The mutual exclusion properties holds if there are not states of of the kind <p_2,q_2,_> (non ci devono essere stati dove entrambi i processi vogliono assegnare il turn a 1 o 2)
  - building the new state diagram => only 4 states

1 CHECKING CORRECTNESS (2)

- To check the correctness we have to check that the three properties hold
  - mutual exclusion is satisfied
    - there are not scenarios including states of the kind <p_2,q_2,_>
  - freedom of deadlock is satisfied
    - if some processes are trying to enter the CS (executing the await), then one eventually succeeds
  - freedom of starvation is **not satisfied**
    - a process that wants to enter in CS (pre-protocol) can starve while waiting a process which engaged an infinite loop inside its NCS (where there is not the progress property)

La variabile

- Turn function as permission resource to enter the CS
  - if the process holding the permission remains indefinitely in its non-critical section, the other process will never receive the resource and will never enter its critical section
    - both processes sets and tests a single global variable: if one process dies, the other is blocked.
- Therefore the solution is *not correct*

2 SECOND ATTEMPT

- We introduce separate want variables intended to indicate when a process is in its CS (intenzione di p e q di entrare nella propria sezione critica)

aspetto che l'altro processo non voglia entrare nella propria sezione critica

Second attempt	
boolean wantp ← false boolean wantq ← false	
p	q
loop forever p1: *non-critical section* p2: await wantq = false p3: wantp ← true p4: *critical section* p5: wantp ← false	loop forever q1: *non-critical section* q2: await wantp = false q3: wantq ← true q4: *critical section* q5: wantq ← false

Pre-Protocol di p

Post-Protocol di p

Pre-Protocol di q

Post-Protocol di q

Se sono in p2 e q2 contemporaneamente, significa che entrambi non vogliono entrare, ma in realtà non hanno ancora impostato a true la loro voglia di entrare (quindi alla fine i want diventano entrambi contemporaneamente a true e quindi non viene soddisfatta la mutua esclusione)

- Mutual exclusion: not having scenarios with p_4 and q_4

2 SECOND ATTEMPT ABBREVIATED

- Removing irrelevant statements..

Second attempt: Abbreviated	
boolean wantp ← false boolean wantq ← false	
p	q
loop forever p1: await wantq = false p2: wantp ← true p3: wantp ← false	loop forever q1: await wantp = false q2: wantq ← true q3: wantq ← false

- Mutual exclusion: not having scenarios with p_3 and q_3

CORRECTNESS OF THE 2nd ATTEMPT

- By constructing the state diagrams it is simple to see that the mutual exclusion property is not satisfied
  - the state <p_3,p_3,`true`,`true`> is reachable.
- The problem is due to the non-atomicity of the pre-protocol and post-protocol

3 THIRD ATTEMPT...

- Previous problem can be solved by considering the await statement part of the CS and moving the assignment before the await

(ora viene swappato rispetto a prima: ora si mette a true la propria intenzione e poi si controlla che quello contro non voglia entrare: però ora ho deadlock)

i due processi si bloccano: i due want non diventeranno mai false perchè entrambi si bloccano su await

Third attempt	
boolean wantp ← false boolean wantq ← false	
p	q
loop forever p1: *non-critical section* p2: wantp ← true p3: await !wantq p4: *critical section* p5: wantp ← false	loop forever q1: *non-critical section* q2: wantq ← true q3: await !wantp q4: *critical section* q5: wantq ← false

Pre-Protocol di p

Post-Protocol di p

Pre-Protocol di q

Post-Protocol di q

- Correctness of the 3rd attempt
  - the mutual exclusion problem is solved, however the solution suffers of possible deadlocks
    - both processes are trying to enter in their CS but neither will ever do
  - more precisely, this is a *livelock*

4 FOURTH ATTEMPT...

- Deadlock occurs when both processes simultaneously insist on entering their CS
  - we can solve the problem by requiring a process to give up its intention to enter its CS if it discovers that it is contending with the other process

Fourth attempt

boolean wantp ← false
boolean wantq ← false

p	q
loop forever	loop forever
p1: *non-critical section*	q1: *non-critical section*
p2: wantp ← true	q2: wantq ← true
p3: while wantq	q3: while wantp
p4: wantp ← false	q4: wantq ← false
p5: wantp ← true	q5: wantq ← true
p6: *critical section*	q6: *critical section*
p7: wantp ← false	q7: wantq ← false

Pre-Protocol di p

Post-Protocol di p

Pre-Protocol di q

Post-Protocol di q

Controllo se l'altro processo vuole entrare in CS: se si, tolgo la mia intenzione (per far eseguire all'altro la sezione critica) e poi la rimetto.

- Correctness of the 4th attempt
  - the livelock is solved, but there is still the possibility of starvation, in the case of perfect interleaving in executing while...

(entrambi vedono sempre che l'altro vuole entrare in CS e quindi nessuno dei due mai va in CS perchè avviene in perfetto interleaving l'esecuzione del while)

5

DEKKER'S ALGORITHM

- The problem can be solved with a simple variation of the 4th algorithm, requiring that *the right to insist* on entering - instead of the right to enter - is explicitly passed between the processes by means of the variable turn
  - this solution was found for the first time by the dutch mathematician T.J. Dekker, in 1965
  - Dekker's algorithm for solving the CS problem can be then considered a combination of the first and fourth attempt

5 A SOLUTION: DEKKER'S ALGORITHM

Dekker's algorithm

Algo di Dekker funziona solo per due processi (bisogna estendere il ragionamento a 2+ processi)

```
boolean wantp ← false
boolean wantq ← false
integer turn ← 1
```

p	q
loop forever	loop forever
p1: *non-critical section*	q1: *non-critical section*
p2: wantp ← true	q2: wantq ← true
p3: while wantq	q3: while wantp
p4: if turn = 2	q4: if turn = 1
p5: wantp ← false	q5: wantq ← false
p6: await turn = 1	q6: await turn = 2
p7: wantp ← true	q7: wantq ← true
p8: *critical section*	q8: *critical section*
p9: turn ← 2	q9: turn ← 1
p10: wantp ← false	q10: wantq ← false

Se è il turno di q, lascio entrare q in CS (questo risolve il problema di Starvation della prima soluzione)

p vuole entrare in CS

se anche q vuole entrare (conflitto), se no salto il while ed entro direttamente

Pre-Protocol di p

Pre-Protocol di q

Dopo che è il turno di p, rimetto la voglia di entrare in CS

Post-Protocol di p

Post-Protocol di q

- Dekker's algorithm is correct
  - it satisfies mutual exclusion, and it is free from deadlock and starvation.

PETERSON'S ALGORITHM

- Other solutions have been proposed through time, improving Dekker's one:
  - Peterson (1981), Manna-Pnueli, Doran-Thomas
- Peterson's solution is more concise, by collapsing the two await statements into one with a compound condition

```
boolean wantp := false;
boolean wantq := false;
integer turn := 1;
```

p	q
loop forever	loop forever
p1: NCS	q1: NCS
p2: wantp := true	q2: wantq := true
p3: turn := 2	q3: turn := 1
p4: await !wantq || turn = 1	q4: await !wantp || turn = 2
p5: CS	q5: CS
p6: wantp := false	q6: wantq := false

Pre-Protocol di p

Post-Protocol di p

Pre-Protocol di q

Post-Protocol di q

Aspetto che l'altro processo non voglia entrare in CS oppure che sia il mio turno

INTRODUCING COMPOUND ATOMIC STATEMENTS

- Dekker's algorithm works on any architecture, providing just load and store as atomic statements.
- Actually, the algorithm and – more generally – the CS problem and mutual exclusion problems can be greatly simplified if we can exploit more complex atomic statements
  - directly provided by the concurrent machine
- Main examples are the *test-and-set*, *exchange*, *fetch-and-add*, *compare-and-swap*
  - test-and-set is an atomic statement defined as the execution of the two following statements with no possibility of interleaving

```
test-and-set (x,r):
< r := x, x := 1 >
```

Test-and-set viene usata per scrivere in una locazione di memoria e restituire il suo vecchio valore come una singola operazione atomica (non interrompibile). Se diversi processi possono accedere alla stessa area di memoria, e se un processo sta eseguendo una test-and-set, nessun altro processo può iniziare un'altra test-and-set finché il primo processo non ha terminato la propria.

  - a commonly used notation for specifying atomicity is to put angled bracket around the group of statements

CRITICAL SECTION WITH TEST-AND-SET

Questa soluzione funziona con N processi

```
integer lock := 0;
```

Variabile Condivisa

p	q
`integer was_locked`	`integer was_locked`
`loop forever`	`loop forever`
`p1: NCS`	`q1: NCS`
`  repeat`	`  repeat`
`p2:  test_and_set(lock,was_locked)`	`q2:  test_and_set(lock,was_locked)`
`p3: until was_locked = 0`	`q3: until was_locked = 0`
`p4: CS`	`q4: CS`
`p5: lock := 0`	`q5: lock := 0`

Ripeto fino a che il valore precedente di Lock è 0: prendo il valore di lock, lo metto in was_locked e poi setto lock ad 1; se was_locked è ad 1 significa che un altro processo è in CS, se a 0, allora nessuno è in CS e posso entrare in CS

Pre-Protocol di p

Pre-Protocol di q

Quando viene eseguita transizione p2/q2, nel LTS mantiene l'esecuzione di test_and_set in maniera atomica

LOCKS

- By exploiting compound atomic statement, we can easily realize a basic *lock* mechanism
  - simple solution for the CS problem
  - protecting critical section of code
- Two stages / atomic operations
  - *acquiring* the lock
  - *releasing* the lock

Sotto certe assunzioni, possiamo considerare 'acquire' un'istruzione atomica (anche se contiene due istruzione: p2&p3 o q2&q3)

```
lock sharedlock;
```

Utilizzabile per N processi

p	q
loop forever p1: NCS p2: acquire(sharedlock) p3: CS p4: release(sharedlock)	loop forever q1: NCS q2: acquire(sharedlock) q3: CS q4: release(sharedlock)

ATOMIC STATEMENT IN JAVA: `synchronized`

Acquire di un Lock (equivalente di un test_and_set)

- Atomic statements in Java can be implemented by means `synchronized` blocks using the same shared `lock` object

```
// process (thread) A
...
synchronized (lock) {
 <statement a>
 <statement b>
 ..
}
...
```

```
// process (thread) B
...
synchronized (lock) {
 <statement c>
 <statement d>
 ..
}
...
```

  – in this way the sequence of actions a.b and c.d are executed without interleaving the individual actions
    - there cannot exist scenarios with sequence like a.c.b.d or c.a.d.b

CS SOLUTION WITH TICKETS: BAKERY ALGORITHM (TIE-BREAKER)

- An alternative solution to CS problem for N-process accounts for introducing a *ticket* to establish the turn of a process

N-process solution for CS with tickets

```
int num ← 1          numero di ticket prossimo da staccare
int next ← 1         prossimo numero da servire
turn[1:n] ← [0,0,0,...]   rotolo dei ticket
```

p[i] Processo i-esimo

```
loop forever
p1: NCS
p2: < turn[i] ← num; num ← num + 1 >   Operazione atomica: prendo il ticket e viene incrementato il numero del prossimo ticket
p3: await turn[i] = next   Aspetta il suo turno
p4: CS
p5: next ← next + 1   Finita la sezione critica, avanza next (prossimo numero da servire)
```

- Potential shortcomings: arithmetic overflow for `turn` and `num`
  - monotonically growing

ABOUT CORRECTNESS: VERIFICATION VS. TESTING

- **Testing** Cercare la presenza di bug in uno scenario, che è quello di test
  - activity looking for *failures*
    - revealing faults
- **Verification** Verificare se una proprietà di correttezza è vera in tutti i possibili scenari di esecuzione
  - checking if the system satisfies its specification
    - "have we built the system *right*?"
- **Validation** Interessa sapere se il sistema è stato costruito come si aspettava lo stakeholder (rispettare requisiti)
  - checking if the system satisfy customer's expectation
    - "have we built the *right system*?"

Fault

- a *manifestation* of an error
  - error = (human) action the produces an incorrect result

Failure

- when a fault is encountered, it may result in a failure
  - a failure may be caused by many faults
  - a fault can cause many failures

LIVESS AND SAFETY TESTS

Si intende anche la Verifica, non solo tests

- Two kinds of tests
  - **Tests of safety** (Qualcosa che deve valere sempre)
    - verify that a class or system behavior conforms to its specification
    - usually taking the form of testing **invariants**
    - typically using *assertions*
  - **Tests of liveness** (Qualcosa che prima o poi diventa qualcos'altro)
    - progress / non-progress tests
    - harder to set up
- Problems & limits of testing
  - typically checking only a subset of the scenarios
  - *"heisenbugs"* phenomenon
    - test code can introduce timing or synchronisation artefacts that can mask bugs

Un heisenbug è un difetto software che scompare o cambia comportamento quando si tenta di studiarlo, osservarlo o correggerlo (debugging). Il nome deriva dal principio di indeterminazione di Heisenberg, poiché l'atto di monitorare il programma (es. inserendo log o usando un debugger) altera l'ambiente di esecuzione, facendo sparire il bug.

TESTING VS. VERIFICATION

- Testing is aimed at verifying that a (correctness) property holds for some selected scenario
  - *testing reveals the presence of errors, not their absence*
- Verification is aimed at verifying that a property holds **for all possible scenarios (traces)**
  - need of *formal* techniques

FORMAL METHODS

- Two principal (classes of) formal techniques:
  - **model checking**
    - where verification is done by generating one by one all the states of the systems and by checking the properties state by state
    - can be automated by model checkers tools
  - **inductive proofs of invariants**
    - invariant properties are proved by *induction* over the states of the system
    - can be automated by tools called deductive systems
- Both techniques rely on some kind of *formal language / calculus to specify correctness properties*

CORRECTNESS PROPERTIES IN PROPOSITION CALCULUS

- With propositional ~~calculus~~ logic, correctness properties are expressed as *logic formulae* that must be true in order to verify the property in some state of the system
  - formulae are assertions obtained by composing propositions through logic connectors
    - and, or, not, implications, equivalence
- In our case propositions are about *the values of the variables* and *of the control pointers* during an execution of a concurrent programming
  - e.g. given the boolean variable `wantp`, an atomic proposition (assertion) `wantp` is true in a certain state if and only if the value of the variable `wantp` is true in that state
- Each label of a statement of a process will be used as an atomic proposition whose interpretation is "the control pointer of that process is currently at that label"
  - e.g. p_1 proposition asserts that the control pointer of the process p is at the label p1

Problema: la logica proposizionale non è adeguata per i nostri scopi perchè non ci permettono di esprimere, dato il programma, proprietà che si esprimono nel tempo (riguardante la dinamica del tempo)

AN EXAMPLE: MUTUAL EXCLUSION

Third attempt	
boolean wantp ← false boolean wantq ← false	
p	q
loop forever p1: *non-critical section* p2: wantp ← true p3: await !wantq p4: *critical section* p5: wantp ← false	loop forever q1: *non-critical section* q2: wantq ← true q3: await !wantp q4: *critical section* q5: wantq ← false

Con la formula logica proposizionale, non riesco a catturare il SEMPRE

- Formula $p_4 \wedge q_4$
  - is true if both control pointers of the processes are in the critical section
- if it exists some state in which this formula is true, then it means that *the mutual exclusion property is **not** satisfied*

> dually, a program *satisfies the mutual exclusion property* if the formula $\neg(p_4 \wedge q_4)$ is true for *every possible state of every scenario*

TEMPORAL LOGIC

- Processes and systems change their state over the time, and then also the interpretation of formulae about their state can change over the time
  - > we need a formal language/calculus that would take this aspect into the account (cosa che la logica proposizionale non fa: non considera l'evoluzione del tempo)
  - > *temporal logic* is one of the most basic and popular one
    - *Amir Pnueli, "The Temporal Logic of Programs", 18th Annual Symposium on Foundations of Computer Science, 1977*
- The **temporal logic** is a formal logic obtained by adding temporal operators to propositional or predicate logic → Introducono degli operatori che permettono di esprimere proprietà di sistemi che evolvono nel tempo
  - **Linear Temporal Logic (LTL)**
    - ~~to express~~ descrivere/esprimere properties that must be true (at a state) for *every possible scenario*
    - linear / discrete model of time
  - **Branching Temporal Logics**
    - to express properties that must be true in some or all scenarios starting from a state
    - an example: **CTL (computational tree logic)**

LTL: TEMPORAL OPERATORS

Vedremo LTL solo con logica proposizionale

- LTL is based on two basic temporal operators: *always* and *eventually*
  - **box** or **always** temporal operator: □ **p**

Proprietà sempre vera

    - the formula □ **p** is true in a state s_i of a computation if and only if the formula p is true in *all* state s_j with j >= i

Proprietà p deve essere vera in tutti i possibili stati dallo stato s_i in poi, per ogni scenario

      - synonym: □ **p** = **G p** (Globally p)
    - the always operator can be used then to specify *safety properties*, because it specifies what must be always be true

Proprietà vera almeno una volta

  - **diamond** or **eventually** temporal operator: ◇ **p**
    - the formula ◇ **p** is true in a state s_i of a computation if and only if the formula p is true in *some* state s_j with j >= i

Per ogni scenario, deve esistere almeno uno stato in cui p sia vera

      - synonym: ◇ **p** = **F p** (Finally p)
    - the eventually operator is used to specify *liveness properties*, because it specifies something that eventually be true

BASIC PROPERTIES

- Reflexivity Riflessività
  - $\Box p \rightarrow p$
  - $p \rightarrow \Diamond p$
- Duality Dualità
  - $\neg \Box p \rightarrow \Diamond \neg p$ Se non è sempre vero p, allora prima o poi p diventerà falso
  - $\neg \Diamond p \rightarrow \Box \neg p$ Se non è vero che prima o poi succederà p, allora è sempre vero che p è falsa
- Sequences of operators
  - $\Diamond \Box p$ Prima o poi esiste uno stato da cui p è sempre vera
  - $\Box \Diamond p$ Sempre vera che prima o poi si verifichi p (si vuole che p diventi vera più di una volta, non solo una volta: caso di loop)

DEDUCTION WITH TEMPORAL LOGICS

- Temporal logic is a formal system of deductive logic with its own axioms and rules of inference
  - it can be used to formalise the semantics of concurrent programs and used to rigorously ~~prove~~ dimostrare correctness properties of programs
- Example of theorems in TL
  - $(\Diamond \Box p \wedge \Diamond \Box q) \rightarrow \Diamond \Box (p \wedge q)$ is true
  - $(\Box \Diamond p \wedge \Box \Diamond q) \rightarrow \Box \Diamond (p \wedge q)$ is false Perché ci può essere un caso in cui p è vera e q falsa o viceversa

SPECIFYING SAFETY PROPERTIES

- Box operator can be used to specify safety properties
  - as properties that must be always true
- □ **p**, where **p = ¬ q** and **q** is the description of a *bad* state
  - an example: mutual exclusion in CS problem

First attempt	
Integer turn ← 1	
p	q
loop forever p1: *non-critical section* p2: await turn = 1 p3: *critical section* p4: turn ← 2	loop forever q1: *non-critical section* q2: await turn = 2 q3: *critical section* q4: turn ← 1

  - mutual exclusion property: $\Box \neg (p_3 \wedge q_3)$ Per ogni scenario, in tutti gli stati deve valere la formula

SPECIFYING LIVENESS PROPERTIES

- Diamond operator can be used to specify liveness properties
  - as conditions that eventually will be true
- ◇ **p**, where **p** is the description of a *good* case
  - an example: *progress property* (*no starvation*) in CS problem

First attempt	
Integer turn ← 1	
p	q
loop forever p1: *non-critical section* p2: await turn = 1 p3: *critical section* p4: turn ← 2	loop forever q1: *non-critical section* q2: await turn = 2 q3: *critical section* q4: turn ← 1

se la proprietà è vera, in questo caso, significa che deve rispettare la proprietà solo nello stato corrente (cioè rispetta solo una volta)

se p2 è vera allora prima o poi p3 vera

  - progress property for *current state*: $p_2 \rightarrow \Diamond p_3$

In attesa di entrare in sezione critica, prima o poi ci entra

  - progress property *for all states*: $\Box (p_2 \rightarrow \Diamond p_3)$

con box, significa che in qualsiasi stato se capita che vale p2 allora prima o poi p3

NEXT OPERATOR

- Besides always and eventually, another unary operator is defined in LTL, called next: ○φ

  Vera, in quello stato, se p è vera nello stato successivo, in ogni scenario possibile

  – ○φ is true if property φ holds in next state, in every possible scenario
    - also represented by X, i.e. X φ (from neXt)

BINARY OPERATORS

- *Always* and *Eventually* are unary operators. An example of useful and frequently used binary operator is *Until*
  - **Until** operator: **p U q** Prima o poi deve verificarsi q, ma prima che si verifichi q, p deve essere vera (sarebbe p eventually q: cioè q prima o poi diventa vera)
    - p U q is true in a state s_i if and only if q is true in some state s_j, j>=i and p is true in all state s_k, i<=k<j.
    - That is: eventually **q** becomes true and that **p** is true until that happens
  - **Weak-Until** operator: **p W q** (se q non si verifica mai, p è sempre vera)
    - like Until operator, but formula q is not required to become true eventually. If it does not, p must remain true indefinitely
    - **p W q** = as long as q is false, p must be true

REMARKS

- Always and Eventually operators can be defined in term of until: Con until, posso generare always ed eventually
  - $\Diamond p = true\ U\ p$ (eventually A becomes true)
  - $\Box p = \neg \Diamond (\neg p)$ (A always remains true) p sempre vera è equivalente a dire che non è vero che esiste in futuro uno stato dove p è falsa

p deve diventare vera prima che lo diventi q: 'not q until p' (quando 'p' diventa vera, allora 'q' può diventare vera (diventa vera prima 'p' e poi 'q'): non può farlo prima perchè 'p' non è vera e 'q' deve rimanere falsa per definizione di Until)

OVERTAKING

- Consider the following scenario in the CS problem

$$try_p, \underbrace{try_q, CS_q, try_q, CS_q, ..., CS_q}_{1000\ times}, CS_p$$

q entra in CS molte più volte di p

- It's not an example of starvation...
  - it is true that $\Diamond\, CS_p$ p entra in sezione critica solo una volta su 1000
  - > but it's evident too that freedom from starvation can be a very weak property!
- in some cases we want to ensure that a process would enter its critical section within a reasonable amount of time

Logiche temporali posso essere usate anche per specificare proprietà articolate

K-BOUNDED OVERTAKING PROPERTY

- ***k*-bounded-overtaking property**

  Se un processo 'p' fa richiesta di entrare in CS, un altro processo può entrarci al massimo k volte prima di 'p'

  – from the time a process p attempts to enter its critical section, another process can enter at most *k* times before p does

  – Example: 3-overtaking

    - $try_p, try_q, CS_q, try_q, CS_q, try_q, CS_q, CS_p$

- The property can be expressed by the weak until operator W

  – example: 1-bounded overtaking

$try_p \rightarrow (\neg CS_q)\ W\ (\ CS_q\ W\ ((\neg CS_q)\ W\ CS_p\ ))$

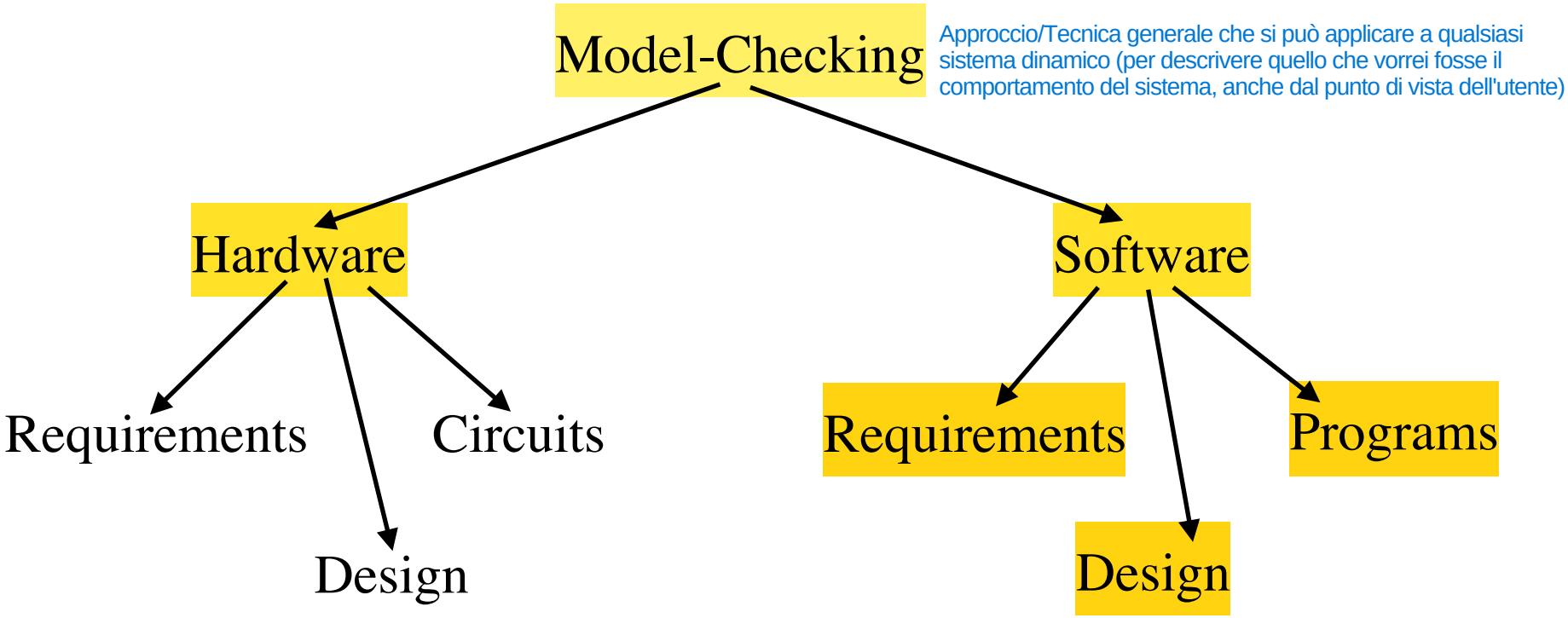
any position satisfying try_p(that is P is in try_p) is followed by an interval in which Q is not in cs, followed by an interval in which Q is in cs, followed by an interval in which Q is again not in cs, which can be terminated only by a state in which P is in cs. It therefore implies that from the time P starts waiting at try_p, there can be at most one continuous interval in which Q is in cs, before P is admitted to cs.

VERIFICATION TECHNIQUES (1/2): MODEL-CHECKING

generazione di tutti i possibili percorsi (nel nostro caso)

- **Model checking** is the most important and used technique for automatically checking correctness properties of concurrent systems
  - invaluable conceptual and practical tool for software engineers
- Strategy based on *exhaustively searching the entire state space* of a system and verify if certain properties are satisfied
  - properties as predicates on a system state or states, expressed as a logical specification such as propositional temporal logic formula
  - if the system satisfies the property, the model checker generates a *confirmation* response
    - otherwise, it produces a *trace* (counterexample) => useful also to identify bugs, not only to prove correctness
- SW vs. HW model checking
  - can be applied also to hardware
  - e.g. Intel adopting Model-Checking after the Pentium Bug in 1994
  - used in mission critical software systems
    - e.g. NASA after Mars Polar Lander incident in 1999

MODEL-CHECKING APPLICATIONS



- Program model checking
  - application of the model-checking techniques to software systems
    - in particular to the final implementation
    - discovering software defects

DEALING WITH THE STATE-SPACE EXPLOSION PROBLEM

- The big problem of model-checking technique is the **size of the state space**
  - how to manage graph of millions of states? Is it feasible ?
- State-of-the art techniques
  - applying rules to reduce the number of states
    - using variables that can be modelled by a limited number of values
  - incremental construction of the whole graph
    - exploring only reachable state of an execution
    - checking the truth of a correctness specification as the incremental diagram is constructed, stopping the construction is a falsifying state is found
  - *symbolic* model checking
    - working with set of states

SPIN AND PROMELA

- **SPIN** is a widely used model-checker used in both academic research and industrial software development
  - extremely efficient
  - used in modelling and designing concurrent and distributed systems in general
- **PROMELA** is the language that is used in Spin to write concurrent programs modelling language Linguaggio per descrivere i processi (molto simile a pseudo-codice)
  - limited number of constructs intended to be used to build models of concurrent systems

AN EXAMPLE: DEKKER IN PROMELA

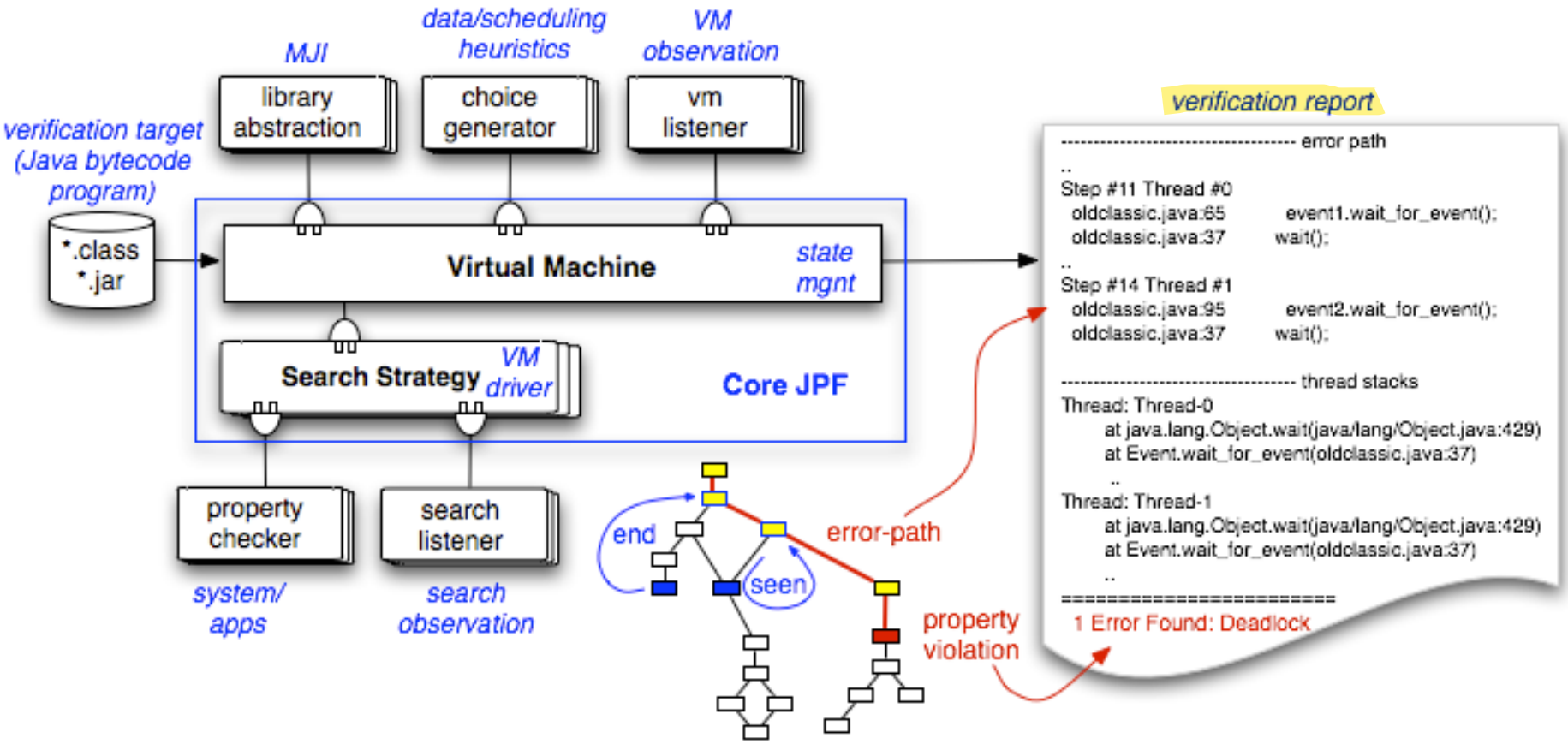
```
bool    wantp = false, wantq = false;
byte    turn = 1;

proctype p() {   Processo p
    do ::
        wantp = true;
        do :: !wantq -> break;
           :: else ->
            if :: (turn == 1)
               :: (turn == 2) ->
                   wantp = false;
                   (turn == 1);
                   wantp = true
            fi
        od;
        printf("LOG: p in CS\n");
        turn = 2;
        wantp = false
    od
}

proctype q() { /* similar */}  Processo q

init {
  atomic {
    run  p
    run  q
  }
}
```

JAVA PATH FINDER (JPF)

- JPF is a recent model-checker specialized for the verification of programs written in Java
  - developed by NASA, used for critical software
  - open-source project
    - http://javapathfinder.sourceforge.net/

- JPF is a special JVM executing programs theoretically along all possible scenarios (execution paths), checking for property violations
  - deadlocks, uncaught exceptions, etc
  - If it finds an error, JPF reports the whole execution that leads to it

JPF MODEL OF OPERATION



VERIFICATION TECHNIQUES (2/2): INDUCTIVE PROOF OF INVARIANTS

- **invariant**
  - a formula that must be invariably true at any point of any computation
    - e.g. $\Box \neg (p_3 \wedge q_3)$
- Invariants can be proved using **induction** over the states of all the computations:
  - to prove that A is an invariant:
    - prove that A is true in the initial state (*the base case*)
    - assume that A is true in a generic state S *(inductive hypothesis)* and prove that A is true in all the possible state next to S *(inductive step)*
- *Deductive systems*
  - software systems for automated theorem proving

SAFETY AND LIVENESS PROPERTY VERIFICATION

- Safety property are easier to verify
  - a safety property must be true at all states
    - it is sufficient to find a state not verifying the property to complete the verification
  - a liveness property claims that a state satisfying a property will inevitably occur
    - it is not sufficient to check states one by one, it is necessary to check all possible scenarios
    - > it requires more complex theory and software techniques

TEMPORAL LOGIC OF ACTIONS (TLA/TLA+)

Oltre a JPF, sono presenti altri Model Checker

- A formal specification language introduced by Leslie Lamport
  - based on simple discrete math (set theory and predicates)
  - http://research.microsoft.com/en-us/um/people/lamport/tla/tla.html
- Used to specify and verify complex real-world systems
  - es: Amazon Cloud
    - "Use of Formal Methods at Amazon Web Services"
      - http://research.microsoft.com/en-us/um/people/lamport/tla/amazon.html

Logica per descrivere proprietà di correttezza e fare Model Checking

TLA+ SPEC

in modo matematico

- A TLA+ specification describes the set of all possible legal behaviors (execution traces) of a system.
  - can be used to describe both
    - the desired correctness properties of the system (the 'what')
    - the design of the system (the 'how')
- TLA+ objective: make it possible to show that a system design correctly implements the desired correctness properties
  - either via conventional mathematical reasoning
  - by using tools such as the TLC model checker
    - a tool which takes a TLA+ specification and exhaustively checks the desired correctness properties across all of the possible execution traces
- PlusCal (formerly called +CAL)
  - an algorithm language based on TLA+
  - for writing algorithms, just as a programming language is for writing programs
    - a PlusCal algorithm is translated to a TLA+ specification, which can be checked with the TLC model checker

PLUSCAL EXAMPLE

```
--algorithm Peterson {
    variables flag = [i ∈ {0,1} ↦ FALSE], turn = 0;
    process (proc ∈ {0,1}) {
        a0:  while (TRUE) {
        a1:      flag[self] := TRUE;
        a2:      turn := Not(self);
       a3a:      if (flag[Not(self)]) {goto a3b} else {goto cs} ;
       a3b:      if (turn = Not(self)) {goto a3a} else {goto cs} ;
        cs:      skip; \* critical section
        a4:      flag[self] := FALSE;
             } \* end while
    } \* end process
} \* end algorithm
```

Fig. 1. Peterson's algorithm in PlusCal.

Program Counter

$$MutualExclusion \triangleq (\overline{pc}[0] \neq \text{“cs”}) \vee (\overline{pc}[1] \neq \text{“cs”})$$

$$\text{THEOREM } Spec \Rightarrow \Box MutualExclusion$$

BIBLIOGRAPHY

- “Principles of Concurrent and Distributed Programming” - M. Ben-Ari. Addison-Wesley
  - chapter 1, 2, 3, 4
- Further suggested reading & references
  - “What good temporal logic is?” - Lamport, 1983
  - “Transition Systems” - M. Nygaard, 2004
  - “Principle of Model Checking” - Baier & Katoen